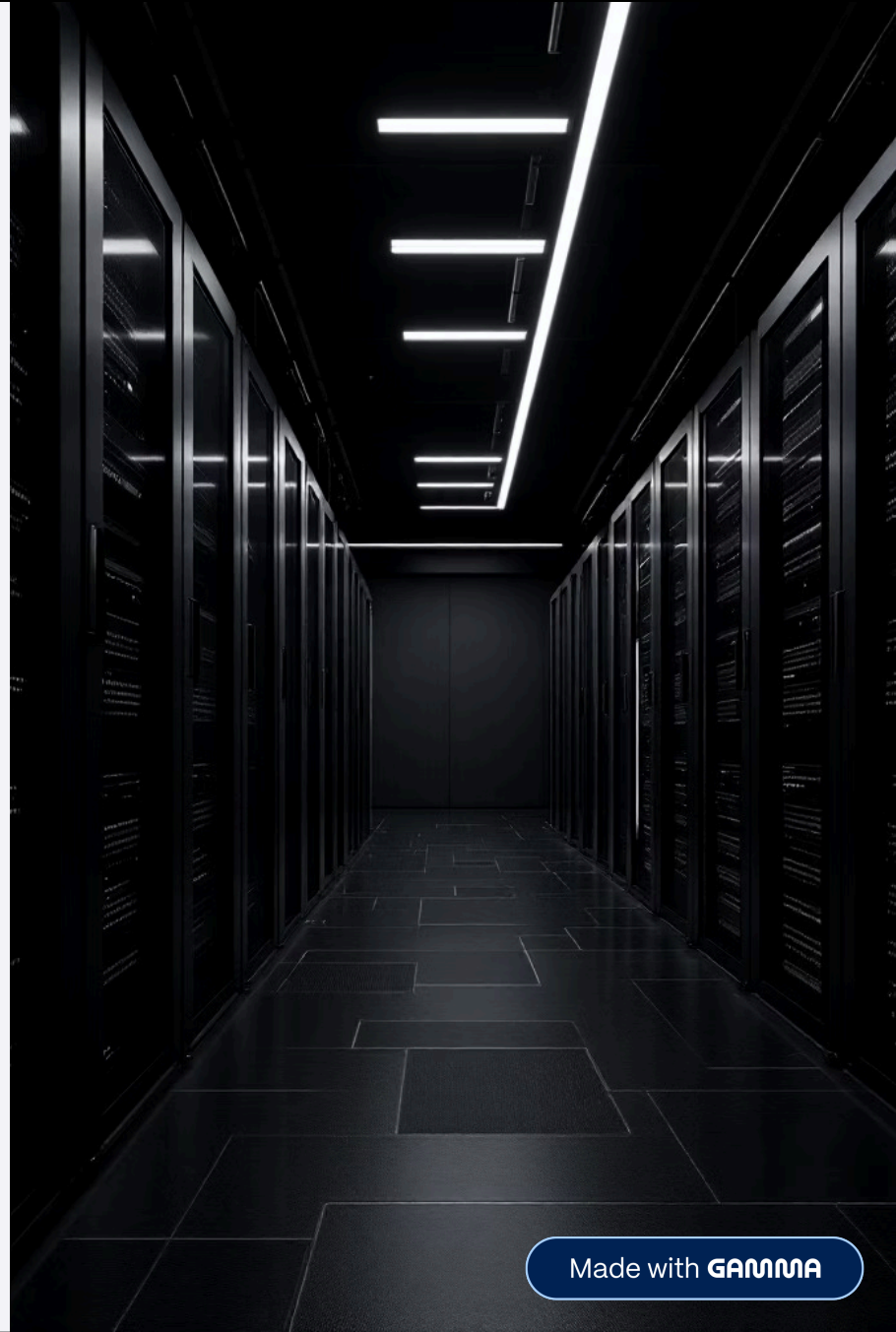
# I/O Systems & Parallel Computing

From Data Transfer to High-Performance Architectures



Made with GAMMA

# Comprehensive Roadmap

01

### I/O Systems Foundation

Communication models, buses, DMA, and modern interface protocols

02

### Parallel Computing Paradigms

Multi-core architectures, SIMD vs. MIMD, and hardware acceleration

03

### Challenges & Future Trends

Parallel programming synchronization and emerging I/O technologies

Made with GAMMA

# What is an I/O System?

The subsystem of a computer that manages communication between the CPU, memory, and external devices. I/O systems provide standardized interfaces for diverse devices while managing data flow, control signals, and error handling.

## Key Roles

- Standardized interface for storage, network, and human interface devices
- Manages data flow and control signals
- Handles errors and device status reporting

## Device Categories

**Block Devices:** Store data in fixed-size blocks (HDDs, SSDs, USB drives). Access is random.

**Character Devices:** Stream of bytes (keyboards, mice, serial ports). Access is sequential.

**Network Devices:** Stream-oriented with packet handling.

### CPU

Initiates requests and processes I/O commands.

### Memory

Buffers data and coordinates with CPU for transfers.

### I/O Controller

Manages device protocols and handles transfers.

### Devices

External peripherals connected via the system bus.

Made with GAMMA

# I/O Communication Models

Three primary methods for CPU interaction with I/O devices, each with distinct trade-offs:

### Programmed I/O (Polling)

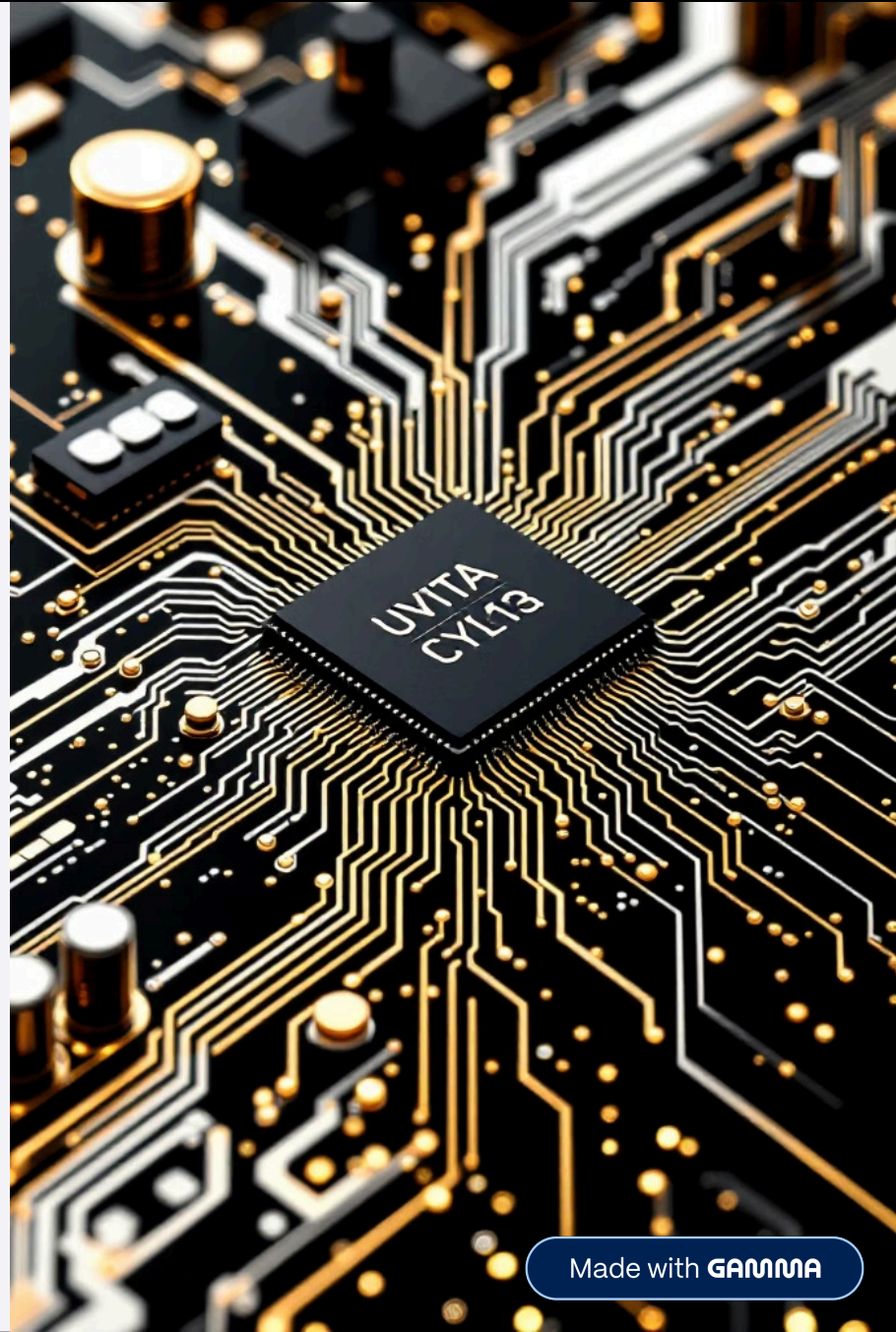
CPU constantly checks device status register. Simple but wastes CPU cycles through busy-waiting.

### Interrupt-Driven I/O

Device sends interrupt when ready. CPU saves context and executes ISR. Efficient but has context switching overhead.

### Direct Memory Access

Specialized controller transfers data without CPU involvement. Highest efficiency for large data transfers.




Made with GAMMA

# System Bus Architecture

A bus is a shared communication pathway connecting computer subsystems. Understanding bus architecture is fundamental to I/O system design.

### Bus Types

- **Processor-Memory Bus:** Short, high-speed, design-specific
- **I/O Bus (Expansion Bus):** Longer, standardized (PCIe, SATA), connects peripherals

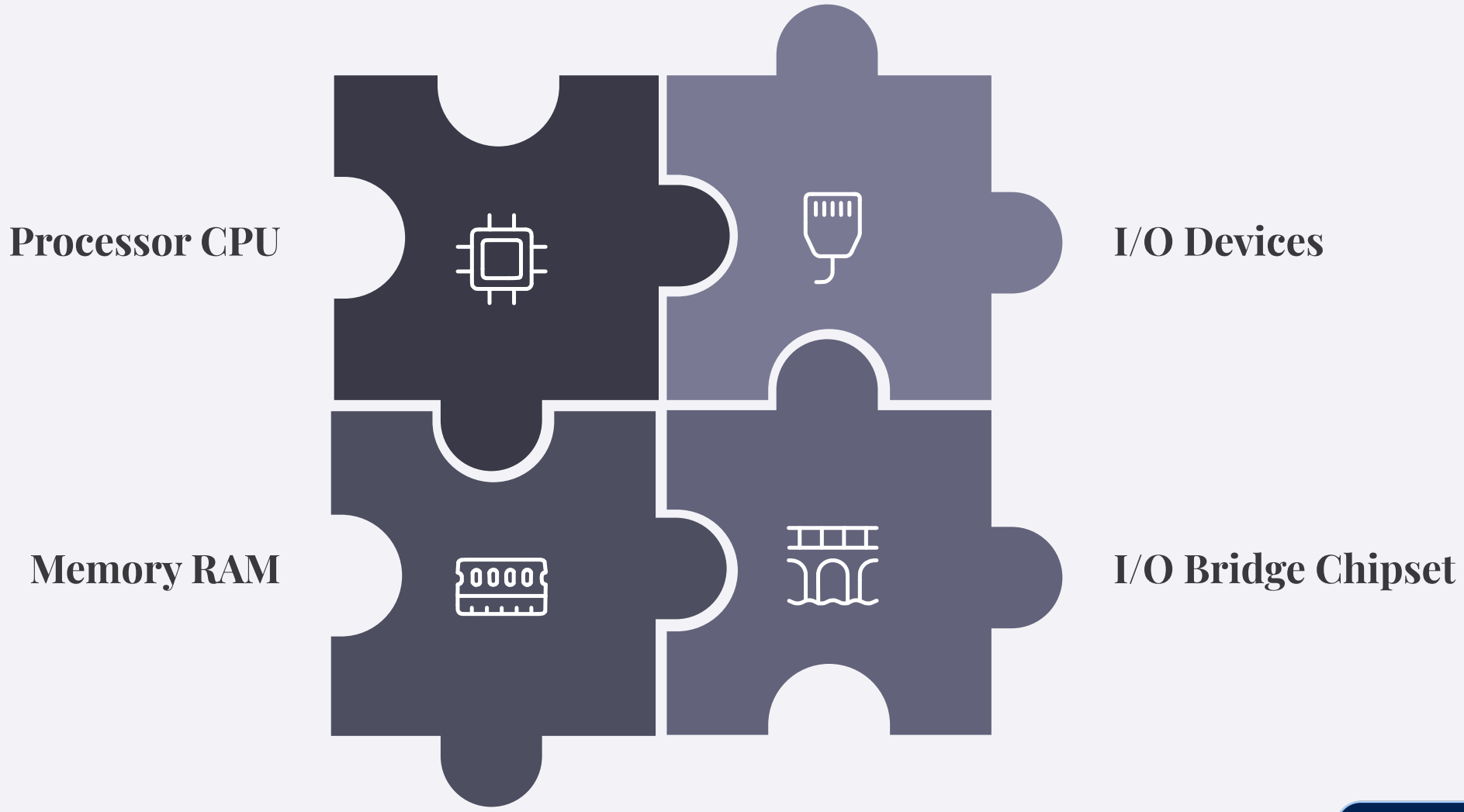
### Bus Lines

- Data Lines: Carry actual data
- Address Lines: Specify source/destination
- Control Lines: Carry commands and signals

### Data Transfer Protocols

**Synchronous:** Uses clock signal. Fast for short distances.

**Asynchronous:** Uses handshaking signals (Req, Ack). Slower but robust for varied devices.




Made with GAMMA

# Direct Memory Access (DMA) Operations

DMA enables specialized controllers to transfer data between devices and memory without CPU involvement, maximizing efficiency for large data transfers.

1

### Setup

CPU programs DMA controller with memory address, byte count, device, and direction

2

### Transfer

DMA controller manages bus access and moves data directly between I/O device and memory

3

### Termination

DMA controller sends interrupt to CPU signaling transfer completion

### Burst Mode

DMA holds the bus for entire transfer, blocking CPU access

### Cycle Stealing

DMA takes bus for one word at a time, interleaving with CPU operations

Made with GAMMA

# Modern I/O Protocols: USB & Thunderbolt

## USB (Universal Serial Bus)

- **Architecture:** Host-controlled, tiered-star topology
- **Evolution:** USB 2.0 (480 Mbps) → USB 3.x (5-20 Gbps) → USB4 (40 Gbps)
- **Protocol:** Polling-based, host asks devices for data
- **Advantages:** Hot-pluggable, power delivery, single driver stack

## Thunderbolt

- **Architecture:** Daisy-chain topology (up to 6 devices)
- **Protocol:** Combines PCIe and DisplayPort into one cable
- **Evolution:** Thunderbolt 3/4/5 (40-80 Gbps, 240W power)
- **Advantages:** Lowest latency, high bandwidth, direct PCIe connectivity

Made with GAMMA

# I/O Bottlenecks & Performance Solutions

Common performance issues include latency, bandwidth limitations, CPU overhead, and bus contention. Strategic solutions address each challenge:

### Caching

Use fast memory (RAM/SSD) to store frequently accessed data, reducing access latency

### Buffering

Smooth data flow between producer and consumer processes

### Interrupt Coalescing

Group multiple interrupts into one to reduce CPU load and context switching

### Asynchronous I/O

Application continues processing while I/O occurs in background

### RAID Arrays

Striping data across multiple disks to increase throughput and reliability

Made with GAMMA

# I/O Virtualization Technologies

Enabling multiple virtual machines to share physical I/O resources through different virtualization approaches:

### Full Virtualization

Hypervisor emulates generic hardware. Excellent compatibility but high overhead due to emulation.

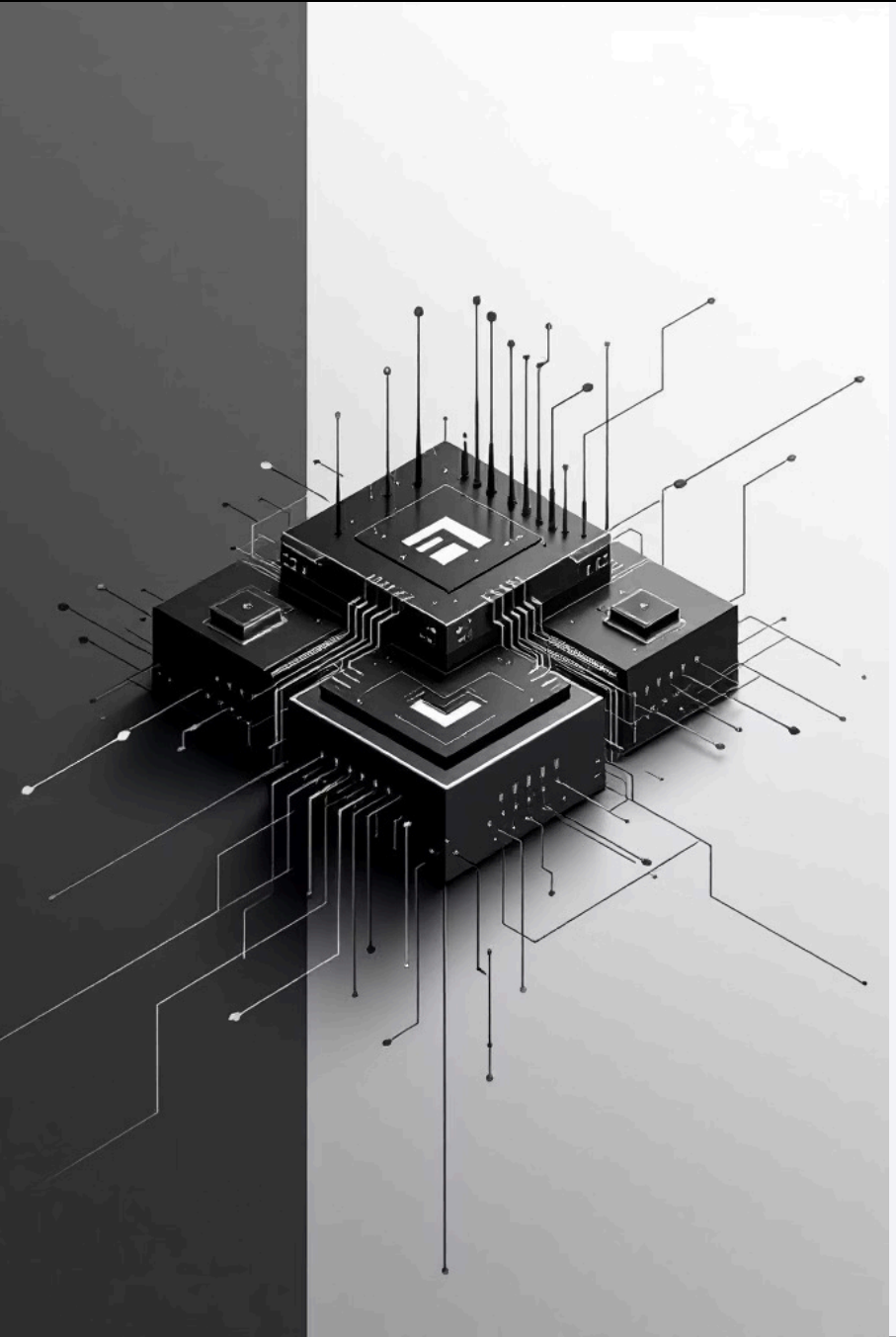
### Paravirtualization

VM uses special drivers (virtio) aware of virtualization. Lower overhead but requires guest OS support.

### SR-IOV

Physical device appears as multiple virtual functions. Near-native performance but requires hardware support.

Made with GAMMA



# Flynn's Taxonomy: SIMD vs. MIMD

Classifying computer architectures by their instruction and data stream organization:

## SIMD: Single Instruction, Multiple Data

A vector instruction operates on arrays of data simultaneously. Implemented in CPU extensions (AVX, NEON), GPUs, and vector supercomputers.

**Pros:** Energy efficient, simple control logic

**Cons:** Requires uniform data structure

## MIMD: Multiple Instruction, Multiple Data

**Shared Memory:** All cores access same RAM. Easy to program but suffers from cache coherency.

**Distributed Memory:** Each node has own memory. Scales well but requires message passing (MPI).

Made with GAMMA